从零开始 • 走向独立闭环

B 组 FPGA 加速器教材

PYNQ-Z2 · 整数 CNN · RTL · AXI/DMA · 板端验证

> **学习目标** 读完并完成练习后，你应能独立读懂 A 组交付，设计、实现、验证和上板一套数值可信的 B 组 FPGA 加速器，并与 C 组完成接口交接。

以 2026-09-25 冻结的 A 组基线接口为案例。面向没有数字电路或 FPGA 基础的读者。

先懂数值和时序，再手算资源，最后逐关完成仿真、实现和板端验收。

版本 1.0 | 竞赛学习用途 | 不宣称已完成 B 组硬件

学习路线与目录

建议顺序：先做概念与纸笔练习，再实现局部 RTL，最后完成系统联调。每章的“自检”用于验证理解。

导读：学完后应能做什么 5

第 1 章 B 组到底负责什么 5

1.1 先讲任务，不从代码开始 . . . 5

1.2 版本比代码更重要 . . . 6

1.3 一份可验收的 B 交付是什么 . . . 6

第 2 章 看懂 PYNQ-Z2：两台机器在一颗芯片里 7

2.1 PS、PL、DDR 与外设 . . . 7

2.2 认识资源的真实含义 . . . 7

2.3 画出自己的硬件框图 . . . 7

第 3 章 数字电路基础：写 RTL 之前先学时间 8

3.1 位、补码和定点数 . . . 8

3.2 组合逻辑、寄存器、时钟 . . . 8

3.3 reset、时钟域、有效握手 . . . 8

第 4 章 把神经网络改写成可寻址的整数计算 9

4.1 卷积的四重循环 . . . 9

4.2 真正手算一层 . . . 9

4.3 其他算子并非“免费” . . . 9

第 5 章 INT8 数值协议：逐位相等的核心 10

5.1 为什么不能直接把浮点乘法换成整数乘法 . . . 10

5.2 最近偶数和负数的坑 . . . 10

5.3 每层尺度和非卷积算子的边界 . . . 10

第 6 章 先算预算，再选择加速架构 11

6.1 把“要快”变成四个数字 . . . 11

6.2 为什么数据搬运会吞掉性能 . . . 11

6.3 两种常见数据流，不存在免费午餐 . . . 12

第 7 章 存储层次、line buffer 与分块 12

7.1 三行卷积为何只缓存两行历史 . . . 12

7.2 tile、halo 与边界 . . . 12

7.3 U-Net skip 的生命周期 . . . 13

7.4 什么时候考虑权重常驻 . . . 13

第 8 章 一个可复用的卷积引擎应怎样思考 13

8.1 MAC、PE、阵列不是三个名字的同一件事 . . . 13

8.2 累加器宽度先证明，再写代码 . . . 14

8.3 重定点与流水对齐 . . . 14

第 9 章 非卷积算子与网络调度 15

9.1 ReLU、Pool、Upsample 的逐位合同 . . . 15

9.2 Concat 与尺度不一致 . . . 15

9.3 层调度表 . . . 15

第 10 章 可综合 RTL 的工程习惯 16

10.1 先写合同，再写 always 块 . . . 16

10.2 处理 ready/valid 的最小示意 . . . 16

10.3 参数化和可重现 . . . 16

第 11 章 PS/PL、AXI 与 DMA：把正确算子变成完整系统 17

11.1 三类 AXI 不要混用 . . . 17

11.2 Stream 中的 valid、ready、last . . . 17

11.3 缓存一致性和缓冲区所有权 . . . 17

11.4 一个 B/C 接口冻结表 . . . 18

第 12 章 验证：从一个乘加到整帧的证据链 18

12.1 参考模型、被测设计、比较器 . . . 18

12.2 必备的定向测试集 . . . 19

12.3 整网、精度和板端四层验收 . . . 19

第 13 章 综合、实现与真实时序 20

13.1 四张报告代表四个不同阶段 . . . 20

13.2 资源率不是唯一 KPI . . . 20

13.3 性能分解与 Amdahl 直觉 . . . 20

第 14 章 上板调试和故障定位 21

14.1 从最小闭环开始 . . . 21

14.2 用“首个失配”缩小搜索空间 . . . 21

14.3 错误处理也是功能 . . . 21

第 15 章 不依赖本书作者，独立做完 B 组的路线图 22

15.1 八个关卡与退出条件 . . . 22

15.2 每周与同学沟通什么 . . . 22

15.3 独立完成能力的最终自测 . . . 23

第 16 章 从空目录动手：工具流程与实验记录 23

16.1 第一次打开 Vivado 前 . . . 23

16.2 先跑一个测试平台 . . . 23

16.3 一个独立 PL 顶层的批量实现模板 . . . 24

16.4 实验记录表要能让别人复现 . . . 24

16.5 第一次建立 Zynq Block Design 的实际顺序 . . . 24

16.6 如何从初学者教程升级到本项目 . . . 25

第 17 章 八个实训：局部可复现，不偷跳整网验收 26

17.1 实训一：从二进制文件读出正确的数 . . . 26

17.2 实训二：一条 3×3 流如何生成窗口 . . . 26

17.3 实训三：一个 MAC lane 到两个 PE . . . 26

17.4 实训四：把 Q24 舍入写成可证明的整数电路 . . . 27

17.5 实训五：非卷积与 skip 的合成陷阱 . . . 27

17.6 实训六：ready/valid 和 AXI-Stream 背压 . . . 27

17.7 实训七：DDR/DMA loopback 后再上单层 . . . 28

17.8 实训八：用一次受控优化完成工程回归 . . . 28

第 18 章 三张计算纸：地址、位宽、吞吐 28

18.1 地址纸：让每个字节都有名字 . . . 28

18.2 位宽纸：为什么“它是 INT8 网络”仍可能需要 64 位 . . . 29

18.3 吞吐纸：算得快与送得上是两回事 . . . 29

附录 A 算术、预算与数据读取练习 30

A.1 纸笔题与答案 . . . 30

A.2 真实首层向量的可复算路径 . . . 30

A.3 你自己写一个最小比较器 . . . 31

附录 B 接口冻结、验证与交付模板 31

B.1 接口冻结记录（复制后填写） . . . 31

B.2 数值和硬件报告模板 . . . 32

附录 C 术语、来源与阅读索引 32

C.1 速查术语 . . . 32

C.2 项目内一手材料 . . . 32

C.3 官方外部资料 . . . 33

导读：学完后应能做什么

本书面向没有 FPGA 经验、但要协助 B 组同学的人。目标不是让读者复述术语，而是独立完成一条可审计的工程链：读懂 A 组整数模型包，算清板卡预算，选择数据流，写并验证卷积和非卷积模块，联通 PS/PL 与 DDR，拿到综合、时序和板级数值证据，再与 C 组交接。教材不附一份声称已完成的 B 组整网 RTL 工程；每一章给出足以独立搭建它的原理、局部例子、检查办法和失败定位。

阅读时把陈述分成四类：**文件事实**可用本项目哈希和程序核查；**手算/理论估算**用于筛选方案；**示例假设**只是演练；**板级实测**必须由真正的 RTL、实现报告和板卡日志产生。把前三类写成“已经上板”是工程错误。本文以 2026-09-25 可交付的 ISIC 2016 冻结基线作接口例子；后续 A 组候选仍在评估，不能自动替代它。[P1][P2]

学习顺序是：第 1-5 章建立数值和硬件语言，第 6-10 章做架构与 RTL，第 11-14 章做接口、验证和上板，第 15 章把所有步骤连成闭环。每章末的“自检”应自己先做，再看答案。没有板卡时可以做到软件参考、RTL 仿真和资源估计；**不能**把这三个结果写成实板通过。

> **起点检查** 请先确认自己能区分“模型权重文件”“描述如何解释它的规格文件”“对照输出的黄金向量”“FPGA 配置文件 bitstream”。它们是四类文件，不可互换。

第 1 章 B 组到底负责什么

1.1 先讲任务，不从代码开始

给一张皮肤镜照片，A 的网络输出每个像素属于病灶的 logit；转换为二值掩膜后，C 负责把掩膜映回原图并展示。这是**分割**，不是“黑色素瘤/痣”的诊断分类，也没有微小/中/大三个输出类别。面积分组只用于统计精度。B 的职责是把 A 定义的整数网络，变为 PYNQ-Z2 可运行、可量测、数值可信的 PL 加速器；B 与 C 要共同冻结 DDR、DMA、寄存器和帧协议。[P1]

一条帧的数据链可写成：照片 → C 的 RGB/letterbox → 首层 INT8 输入 → DDR/AXI → B 的 15 个卷积及池化/上采样/拼接 → INT8 logit → C 的阈值、去补边、回映与显示。**后处理是否采用仍未裁定**，不要在 B 的冻结接口中悄悄加入连通域规则。[P2]



图 1 教学用数据流：真实工程中的物理接口、缓冲与寄存器由 B/C 冻结。

1.2 版本比代码更重要

一个结果至少绑定四项：checkpoint_sha256、model_spec.json 的 SHA256、参数二进制文件 SHA256、输入样本/预处理版本。旧的 deliverables/role_a/ 是可供 B/C 并行开发的基线；runs/ 是实验区。若拿实验权重搭配旧量化包，形状即使相同，数值也没有意义。B 在启动前、C 在加载前、联调记录里都应核对版本。[P1]

物件	谁产生	B 应核对	不能由它证明
model_spec.json	A	层序、shape、尺度、偏移、哈希	RTL 已实现
weights_int8.bin/bias.bin	A	长度、哈希、按偏移读出的元素	板端精度
acc_golden.bin/output_golden.bin	A	单层小块逐位输出	全网一致
bitstream 与实现报告	B	器件、时钟、资源、WNS	应用端显示正确
帧日志与掩膜	B/C	帧号、版本、输入/输出哈希	医疗诊断价值

1.3 一份可验收的 B 交付是什么

最少应包含：源码与版本、可重复的仿真脚本/黄金向量、模块接口/寄存器表、综合与布局布线报告、目标时钟与实际时序、bitstream、板端逐层或逐帧数值对照、吞吐/延迟/功耗测量条件、与 C 的错误/复位约定。一个“能亮灯”的演示无法代替这些证据。数值和速度都要注明测量层级：仿真、综合估计、实现后估计或板端实测。

> **自检** 如果 A 发布新 best.pt，B 能直接沿用旧 weights_int8.bin 吗？答案：不能；需要 A 重新校准/导出并核对规格与黄金向量，B 再走回归门槛。

第 2 章 看懂 PYNQ-Z2：两台机器在一颗芯片里

2.1 PS、PL、DDR 与外设

Zynq 把处理系统 **PS**（Arm CPU、DDR 控制器等）和可编程逻辑 **PL**（FPGA 逻辑）放在一颗芯片上。PYNQ-Z2 使用 XC7Z020，板上 512 MB DDR3；器件有 630 KB 块 RAM、220 个 DSP slice。注意“630 KB”是器件的标称资源，尚要给 DMA/FIFO/其他 IP 留空间，不能把它全部许诺给 CNN。板卡手册还列出了时钟、复位、电源和接口；实验应以所用板的修订版为准。[S1]

PS 擅长复杂控制、文件系统、预处理和显示；PL 擅长规则的并行数据通路。DDR 物理连接到 PS 的内存控制器，PL 可经 AXI 高性能端口访问。传一次大张量并非“把 Python 数组扔进 FPGA”：要规定物理地址、缓冲区、缓存一致性、传输长度、完成同步与失败恢复。[S2]

2.2 认识资源的真实含义

LUT 是组合逻辑构件，FF 在时钟边缘存一位，BRAM 是片上存储块，DSP 有乘法/加法等专用路径。它们不是可无限拆分的货币：一个 8 位乘法是否占 DSP、一个小数组是否推断为 BRAM、跨层共享后能否闭合时序，都要看综合和实现报告。采用 DSP 比例越高并不必然越快；布线、时钟与访存可能先卡住。[S3]

本模型单个最大 INT8 特征张量有 1,572,864 字节，约 1.50 MiB；即使把 630 KB 按 645,120 字节乐观理解，也放不下这一张。权重 121,760 字节，加偏置 1,412 字节；多个 skip 特征还要活到解码阶段。因此从第一天就要设计 DDR/分块/流式存储，不能等 RTL 写完才发现容量不够。[P1][S1]

2.3 画出自己的硬件框图

纸上先画五个盒子：PS 软件、DDR 缓冲、DMA/AXI、PL 卷积引擎、PL 控制/状态。每条箭头标字节宽、方向、地址来源与节拍。PS 写控制寄存器，DMA 从 DDR 读入；PL 处理后经 DMA 写回；PS 等完成并同步缓存。逐层方案可能让每层来回 DDR；融合方案则尽可能在片上流过邻接算子，两者在简单性/DDR 流量/资源上不同。

> **自检** 为什么“512 MB DDR”不能代替 line buffer？答案：容量虽大，访问延迟与带宽、地址变换和并发仲裁约束远高于片上 BRAM；滑窗每时钟要重用相邻行，必须设计局部缓存或接受巨量重复读取。

第 3 章 数字电路基础：写 RTL 之前先学时间

3.1 位、补码和定点数

8 位无符号数表示 0…255；8 位补码有符号数表示 -128…127。本项目的量化有效范围是 **-127…127**，零点 0，故 -128 虽可在字节里出现，却应视为不符合当前数值协议。两个有符号 8 位相乘，完整结果至少需要 16 位；多个乘积求和以及偏置还需更宽累加器。RTL 里显式写 signed、扩展位宽和截断点，不能指望编译器猜对。[P1]

例如十进制 -3 的 8 位补码是 11111101：3 为 00000011，按位取反加 1 得 11111101。把此字节错误当无符号数会读成 253。一个乘法器输入未签名，整层结果就会系统性失配。

3.2 组合逻辑、寄存器、时钟

组合逻辑的输出只由当前输入决定，寄存器只在触发沿更新。流水线把长的乘加路径分成几级寄存器，缩短单级组合延迟，但增加启动延迟与占用资源。**吞吐**是稳定后每秒完成多少输出；**延迟**是单帧从进到出的时间。流水线可以提高吞吐而不减少单帧首个结果的时钟数。

时钟周期必须覆盖触发器时钟到输出延迟、组合路径、布线、接收触发器建立时间及不确定性；若不满足就是 setup 违例。数据过早变化还会触发 hold 违例。WNS（最坏负裕量）为负说明至少一条检查没过；WNS 非负并不自动证明功能或板端吞吐。时间约束文件 XDC、实现后的 report_timing_summary、跨时钟策略都必须保留。[S4]

3.3 reset、时钟域、有效握手

复位是让状态机、计数器和有效位回到已定义状态，**不是**清空所有 BRAM 的魔法。异步外部信号跨进 PL 时要同步；宽数据跨域可用异步 FIFO，不能把一串比特分别两级同步后当成同一字。若只有一个时钟域，先把它做正确，避免过早引入 CDC 复杂度。

在 ready/valid 流中，只有 valid && ready 的周期发生一次传输；valid=1,ready=0 时，数据及附带帧标志必须保持稳定。这条不变量贯穿算子、FIFO、DMA。写状态机时把“接受输入”和“计算/输出”分清，随机背压仿真可以迅速暴露丢拍和重复拍。[S5]

> **自检** 若某拍 valid=1,ready=0，下一拍上游把 data 改了，是否允许？不允许；下游还没接收旧数据。

第 4 章 把神经网络改写成可寻址的整数计算

4.1 卷积的四重循环

对输出 (o,y,x)，卷积先取该输出通道的 bias[o]，再遍历输入通道 i 和卷积核 (ky,kx)：acc[o,y,x] = bias[o] + Σ_i,ky,kx input[i,y+ky-pad,x+kx-pad] * weight[o,i,ky,kx]。超出图像边界的输入填 0；本基线是 stride 1，3×3 pad 1 或 1×1 pad 0。这个式子解释了 line buffer、权重复用、PE 数量以及累加器位宽。[P1]

NCHW 意思是批次 N、通道 C、行 H、列 W 的索引顺序；一个 [1,3,256,256] INT8 张量在连续布局中，通道 0 的全部像素先于通道 1。OIHW 权重则是输出通道 O、输入通道 I、核行和核列。若 offset = ((n*C+c)*H+y)*W+x，最后的 x 每增加 1 地址加 1 字节。B 可以在 DDR 内约定对齐/行跨度，但读写端必须用同一个协议；从 A 的文件读取时要按原始连续布局解释，不能擅自改排序。[P1]

4.2 真正手算一层

首层是 3→8 的 3×3，输出为 8×256×256。一个输出像素有 3*3*3=27 次 MAC；整层 256*256*8*27=14,155,776 次 MAC；权重 8*3*3*3=216 个，加 8 个偏置，共 224 参数。注意“MAC 次数”是一次乘且加，不应再把它无说明地乘 2 称为同一个量。[P1]

从交付包的**完整**首层向量取输出 (o=0,y=128,x=128)：三个输入通道的 3×3 点积分别为 -4261、21622、-2698；偏置是 -18585。因此 acc = -18585-4261+21622-2698 = -3922。首通道 multiplier_q24=13370，乘积 -52,437,140，除以 2^24=16,777,216 后按最近偶数舍入为 -3；output_full_golden.bin 该位置也是 -3。此例可用附录中的小程序从二进制直接复算，不需要猜测浮点输出。[P1]

本层另有独立 8×8 小块黄金向量。该块取原始大图左上 8×8，再在**块自身边界**填零卷积；不能把块的右/下边缘直接跟完整 256 图同坐标输出比较。先过 acc_golden.bin，再过 output_golden.bin；前者失败定位乘加/偏置/索引，后者才定位重定点/饱和。[P1]

4.3 其他算子并非“免费”

ReLU 是 max(0,x)；2×2 maxpool 在四个输入中取最大，空间尺寸减半；最近邻上采样把每个像素复制到 2×2 位置；Concat 按通道轴把编码器 skip 放在前、上采样特征放在后。它们算术轻，但地址生成、缓存寿命与尺度转换并不轻。把 ReLU 移到卷积输出之前，或把 Concat 通道次序颠倒，整网都会错。[P1]

U-Net 的编码端为 8→16→32→64 通道，三次池化到 32×32；解码端逐级上采样并拼接 skip，最终 1×1 head 输出单通道 **logit**。全网 15 层 Conv，121,760 个卷积权重；bias.bin 为 1,412 字节，每

个 INT32 偏置占 4 字节，所以有 353 个偏置。参数总数 121,760+353=122,113。这个计算说明，文件字节数要除以每元素字节数，且不能凭首层推全网。[P1]

> **自检** 若某层 Cin=16,Cout=32,H=W=64,K=3，MAC 数是 64*64*32*16*3*3=18,874,368。请再算权重字节数：32*16*3*3=4608 字节（INT8），另有 32 个 INT32 偏置，共 128 字节。

第 5 章 INT8 数值协议：逐位相等的核心

5.1 为什么不能直接把浮点乘法换成整数乘法

量化把实数 r 近似为整数 q = clip(round_even(r/s),-127,127)，反量化约为 r≈q*s。本项目激活使用每张量一个正尺度 s_x，权重每输出通道一个尺度 s_w[o]，零点均为 0。两者相乘的真实单位是 s_x*s_w[o]，因此偏置量化为 b_q[o]=round_even(b_float[o]/(s_x*s_w[o]))。累加器里的整数与实数单位是一致的。[P1]

卷积后还要切到输出尺度 s_y。项目已把比例 s_x*s_w[o]/s_y 预先取整成 m_q24[o]≈ratio*2^24；硬件计算 p=int64(acc)*int64(m_q24[o])，q_y=clip(round_even(p/2^24),-127,127)。真实工程首先要对所有层做数值上界证明：INT8 输入乘积、INT32 累加、INT64 中间积在**实际 spec 最大通道和偏置**下都不溢出，再决定每级寄存器宽度。[P1]

5.2 最近偶数和负数的坑

“最近偶数”只在恰好半格时与普通四舍五入不同：2.5→2，3.5→4，-2.5→-2，-3.5→-4。用截断向零、算术右移或 +2^23 再移位，都不能覆盖负数和半格。可先对绝对值做商与余数：q=|p| >> 24，r=|p| & (2^24-1)；若 r>2^23，或 r==2^23 且 q 为奇数，则 q+=1；最后还原符号并饱和。要单独处理有符号最小值取绝对值的溢出问题。[P1]

演练用 m_q24=2^23（仅为教学假设，不是本项目首层的值）：acc=5 得 2.5→2；acc=7 得 3.5→4；acc=-3 得 -1.5→-2；acc=-5 得 -2.5→-2。真正首层的 m_q24 要读 JSON，不要把练习常量写进 RTL。

5.3 每层尺度和非卷积算子的边界

当前软件参考中，每个量化卷积输出先用 output_scale 还原，再经过 ReLU/Pool/Upsample/Concat；进入**下一卷积**时按下一层自己的 input_scale 重新量化。若两层尺度不同，把前层输出的字节不变地接到后层会错。硬件可以把这一过程合并成纯整数尺度变换，但必须以 golden 回归证明逐位相同。output_full_golden.bin 是该 Conv 自身重定点结果，若后面接 ReLU，文件仍是 **ReLU 前**。[P1]

Head 的输出是 INT8 logit 而不是概率。实际 logit 近似为 q*head.output_scale。原冻结基线阈值 0.5 的 sigmoid 等价于 logit≥0，也就是 q≥0；这只是该版本的规则，不代表所有后续候选沿用相同阈值。对非补边区域计算 Dice，边框补零处要强制背景。[P1]

> **自检** -128 输入、换层忘记重定标、把 output_golden 当作 ReLU 后结果，是三个不同错误。你应分别在输入合法性、层间 golden、ReLU 单元测试里抓到它们。

第 6 章 先算预算，再选择加速架构

6.1 把“要快”变成四个数字

架构讨论前，把目标写成：每帧允许多少秒、时钟目标多少 MHz、可留给 CNN 的 LUT/FF/BRAM/DSP 上限、PS 与 PL 共用 DDR 时可用带宽。若比赛尚未规定帧率，先报“延迟候选区间”，绝不凭空宣称实时。功耗也要说明是板级表计、工具估计还是仅理论。

总卷积量按冻结规格逐层求 Hout*Wout*Cout*Cin*K*K，得到 **675,282,944 MAC/图**。若理想情况下每拍并行 P 个 MAC、时钟 f，理论计算下界 T_compute >= MAC/(P*f)；真实利用率 η≤1 时约 MAC/(P*f*η)，还未计 DMA/启动/非卷积/同步。例如仅作假设：P=32、f=100 MHz、η=0.5，则约 0.422 秒/图。**32 个逻辑 MAC/拍不等于恰好 32 个 DSP、综合达到 100 MHz 或整机只需 0.422 秒。**[P1]

问题	纸面可算	必须报告的实测
计算量	逐层 MAC、并行拍数下界	实际有效吞吐、空闲拍
内存	张量/权重/skip 字节及存活期	BRAM 分配、DDR 读写字节/秒
时间	f、P、η 假设下的候选范围	布局布线后 WNS、板端帧延迟
精度	软件 INT8 和 golden	RTL/板端逐位对比、同口径 Dice

6.2 为什么数据搬运会吞掉性能

朴素实现每次 MAC 从 DDR 读 1 字节输入和 1 字节权重，仅卷积读流量的粗上界就是 2*675,282,944≈1.35 GB/图，更不含写输出；它不是必须发生的下界，因为滑窗和权重可以大量复用。首层有 14,155,776 MAC，但独立输入仅 196,608 字节，权重仅 216 字节，输出 524,288 字节。好的 line/weight buffer 的意义，就是用片上字节换掉重复 DDR 流量。若多个层不融合，至少还要读写中间特征；实际字节数取决于布局、对齐、分块和 skip 策略，必须在设计表中逐层累计。

“峰值一个张量 1.50 MiB”不等于“运行时内存只需 1.50 MiB”：U-Net 编码器的 skip 要留给解码器，双缓冲和 DMA 描述符还占空间。对每个张量写明产生阶段、最后一次使用阶段、是否可覆盖前缓

冲；画一张存活区间图再安排 DDR 地址。若 A 换成 12 通道候选，参数和特征会增加，B 的预算须重新计算，而不是沿用此表。[P2]

6.3 两种常见数据流，不存在免费午餐

输出像素优先：固定输出 (o,y,x)，循环所有输入通道和核位置，适合小量 PE、控制简单；若无缓存会反复读相邻输入和权重。**输出通道并行/权重驻留**：同一输入窗口送多个 PE，各 PE 持有一个或若干输出通道的权重；减少重复读输入，却增加并行乘加器和片上权重/累加资源。还可在层间融合 ReLU/Pool 避免 DDR 往返，但让背压、尺度转换和调试边界更复杂。选型依据是实测瓶颈，不是某种数据流“最先进”。

进行一次设计空间表：行列分别列 P_in、P_out、块高/宽、缓存大小、估计周期、估计 DDR 字节、综合资源、时序 WNS。先用简单可验证的低并行方案当金标准，再逐项优化。每次只改一类参数，完整 golden 回归；这样才能判断收益来自哪里。

> **自检** 若计算下界为 0.42 秒，板端测得 2.5 秒，下一步不是立刻加 PE。先测每层 DMA 时间、PE busy 比例与背压；若计算单元大部分时间等 DDR，加 PE 只会增加闲置硬件。

第 7 章 存储层次、line buffer 与分块

7.1 三行卷积为何只缓存两行历史

以逐行流入的 3×3 卷积为例，形成当前窗口需要当前行和前两行，所以最小历史 line buffer 量约 (K-1)*W*Cin*bytes_per_element。首层 W=256、Cin=3、K=3、INT8 时约 2*256*3=1536 字节，另加移位窗口、FIFO 和 BRAM 粒度。这个公式是**输入重用需求**，不是整层所有缓存需求；输出通道并行时还要权重、累加、结果与控制状态。

像素在第 0 行、第 0 列的 3×3 窗口，顶边与左边的越界元素为 0。不能用前一行的末尾当下一行左邻居；行计数、列计数与通道顺序必须同 DDR 布局一致。随机输入尺寸、行尾背压和帧尾复位测试，比只跑中央像素更能抓错。

7.2 tile、halo 与边界

若希望输出一块 T_h×T_w 的 3×3 stride 1 区域，输入块一般需要 (T_h+2)×(T_w+2)（上下左右各 1 halo），边界处从图外补 0。单层该输入块字节数约 (T_h+2)*(T_w+2)*Cin，输出块约 T_h*T_w*Cout。融合连续两层 3×3 时，第二层的每个 tile 输出会依赖原始输入更宽的 halo；不能仍只给一圈。多 tile 之间 halo 可重复读取，块太小会使重复流量和控制成本升高，块太大又挤占 BRAM。要把 T_h,T_w 当可量测的设计参数。

双缓冲在算当前块时预读下一块，理想情况下可把搬运和计算重叠。它要求两个缓冲不互相覆盖、消费者停止时生产者也能停，并明确 buffer ownership；需要约两倍缓存，未必划算。第一次正确实现时可先单缓冲，再按性能剖析决定是否双缓冲。

7.3 U-Net skip 的生命周期

编码器每一层的输出一方面继续下采样，另一方面在解码器上采样后参与 Concat。若只保留最新特征，解码时已无 skip。因此要把编码器 256、128、64 尺寸的 skip 及其尺度/布局保存；可以放 DDR、分块重算或设计可重用缓存，但每种方案都需资源和时延算术。Concat 不是数值相加：同一像素先输出编码器特征的所有通道，之后才是上采样特征通道，次序必须与训练图一致。[P1]

7.4 什么时候考虑权重常驻

冻结基线量化权重合计 121,760 字节，乍看小于 630 KB，但 BRAM 还有行缓存/FIFO/控制用量，且 BRAM 是离散块，端口数也有限。可以尝试整网权重常驻、逐层加载或分片加载三种方案，比较加载延迟、实现后 BRAM 使用与时序。偏置虽只有 1,412 字节，仍须按层的 bias_offset_bytes 和小端有符号 INT32 解释。不要用文件总长度假定层边界。[P1]

> **自检** 为什么 tile 的计算结果可能全对，却在 tile 边界出现条纹？常见原因是 halo 不足、边界填充位置错或各 tile 对同一边界采用不同量化尺度。保存首个错误的 (层,o,y,x) 和所用 tile 地址可快速定位。

第 8 章 一个可复用的卷积引擎应怎样思考

8.1 MAC、PE、阵列不是三个名字的同一件事

MAC 是乘加运算；PE 是包含若干 MAC、局部寄存器/控制的处理单元；PE Array 是多个 PE 的组织。先定义一个 PE 每拍接收多少输入和权重、产出什么位宽、累加多久、何时送出。若声称 P=64 MAC/拍，要画出 64 对操作数如何到达、加法树与寄存器如何排布、写回是否同速。否则“64 并行”只是未布线的数字。

一个朴素可复用引擎可由 address_generator、line_window、weight_reader、mac_lane、accumulator、requant、output_fifo、layer_controller 组成。可让不同层复用同一数据通路；层参数从冻结的 spec 编成可验证描述符。不要让外部软件随意写核大小或偏移超界：只接受支持的 1×1/3×3、通道范围、地址对齐和版本。

8.2 累加器宽度先证明，再写代码

最保守的绝对值上界可写成 |bias[o]| + Cin*K*K*127*127。此界很松，但可证明不会溢出某个位宽；应对 spec 的每层每输出通道读取 bias，取最大上界，再核查 INT32。若用分组部分和，相加前不能在 16 位截断；若做加法树，逐级位宽随输入数量增长。RTL 中表达式的自动扩展规则容易令人误判，所以在每个乘法、加法、取负和位移处显式规定有符号与目标宽度。综合前先用边界向量和 Python 任意精度参考核对。

示意 SystemVerilog 片段（只说明宽度与时序，不是完整可用模块）：

```systemverilog
logic signed [7:0]  x_q, w_q;
logic signed [15:0] product;
logic signed [31:0] acc_q, bias_q;
always_comb product = $signed(x_q) * $signed(w_q);
always_ff @(posedge clk) begin
    if (rst)             acc_q <= '0;
    else if (start_dot)  acc_q <= bias_q;
    else if (mac_fire)   acc_q <= acc_q + {{16{product[15]}},product};
end
```

这里 start_dot 与 mac_fire 是否可能同拍，必须由状态机契约规定；示意代码选择 start_dot 优先。如果真实设计同拍应同时计入首乘积，需改写，不能只看波形“差不多”。mac_fire 只能在两个操作数实际有效且下游容量允许时产生。最终重定点需 64 位或经证明的更宽中间表达式。

8.3 重定点与流水对齐

卷积输出经过偏置、Q24 乘法、最近偶数舍入、饱和，随后可能 ReLU。每一级流水线都要把 (layer_id,o,y,x,frame_id) 或等价控制标签一同延迟，否则数据值可能正确却写错位置。若乘法器 2 拍、舍入 1 拍，地址流水至少同样延迟。一个可复用 scoreboard 不只比较字节，还比较坐标与帧号。

> **自检** 增加流水寄存器后首个像素正确、行尾错位，多半不是卷积公式错，而是 last/row_end/address 没有随数据相同拍数延迟；再施加随机背压验证标签与数据保持一致。

第 9 章 非卷积算子与网络调度

9.1 ReLU、Pool、Upsample 的逐位合同

在同一正尺度下，ReLU 可以对 INT8 取 max(q,0)；最大池化也可直接比较相同尺度的整数。但“同一尺度”是前提，换层前还要按目标 input_scale 重新量化。池化的 2×2 四个位置不得跨行或跨通道；最近邻上采样只复制同一个量化值到 2×2，但它与下一卷积的尺度转换仍要核对。四条边和奇偶坐标要特别测试。[P1]

9.2 Concat 与尺度不一致

Concat 沿通道轴拼接两个特征，它们来自不同路径，可能有不同浮点尺度。如果硬件打算只传原 INT8 字节并在后层统一解释，必须先验证两路字节使用的尺度是否与软件参考等价；一般不能直接假定。一个安全的初版是按每路已知尺度还原/换算到下一层输入尺度，再依照编码器在前的通道次序存放。整数化可用预计算比例与明确舍入，但须把所有中间截断点写清并用全网 golden 证明。[P1]

9.3 层调度表

让调度器按规格层序处理：encoders.0.* → pool → encoders.1.* → pool → encoders.2.* → pool → encoders.3.* → up/concat → decoder_blocks.0.* → up/concat → decoder_blocks.1.* → up/concat → decoder_blocks.2.* → head。这里 .* 只是概念序列；实际 15 个 Conv 的精确名称、通道和地址都必须从 model_spec.json 生成或冻结，不要手抄不核对。每步记录输入张量地址/尺寸/尺度、输出地址/尺寸/尺度、skip 引用、下一步。完成后做内存存活与覆盖检查。

可把每一层当作“事务”：加载配置和权重、确认输入就绪、执行、写回、核对完成位，再让下一层启动。它便于调试，代价是 DDR 往返较多。优化时把相邻算子融合，仍保留可打开的逐层观察点，否则首个失配难定位。

> **自检** 一个 output_full_golden 与 RTL 同层输出一致，但下一层输入不一致，应先查 ReLU、Pool/Upsample、Concat 次序和尺度变换，而非重写卷积 MAC。

第 10 章 可综合 RTL 的工程习惯

10.1 先写合同，再写 always 块

每个模块先写五件事：输入/输出类型与时钟域，什么时候接受一个事务，什么时候产生结果，复位后状态，出错怎样上报。模块接口要标注有符号、位宽、顺序与 valid/ready 语义。line_window 的合同例如：每接受一个像素，维护行列计数；在一个窗口完整/按边界补零时输出一次 (window,y,x)；下游不 ready 时输入 ready 必须适当拉低并保持当前窗口。

组合逻辑用 always_comb 给每个输出默认值，时序状态用 always_ff @(posedge clk)，避免无意锁存器；同一状态寄存器只由一个时序过程写；同拍更新用非阻塞赋值。for 循环在可综合 RTL 中常意味着复制电路，不等于软件“逐次运行”；动态宽乘法、可变索引和大数组是否映到 DSP/BRAM须看综合报告。[S3]

10.2 处理 ready/valid 的最小示意

```systemverilog
assign in_ready = !out_valid_q || out_ready;
always_ff @(posedge clk) begin
    if (rst) begin
        out_valid_q <= 1'b0;
    end else if (in_ready) begin
        out_valid_q <= in_valid;
        if (in_valid) out_data_q <= transform(in_data);
    end
end
```

这是**单拍缓冲示意**，不是任意复杂算子的完整实现。若 out_valid_q=1,out_ready=0，in_ready=0，寄存器不更新，旧数据保持。真实设计还要保持 TLAST、帧号和错误标志；transform 必须是可综合的组合函数，且满足时序。不能在输出拥塞时继续消费输入却没有地方放数据。[S5]

10.3 参数化和可重现

把宽度、最大通道数、tile 尺寸、并行 lane 数放到编译期参数；把每层尺度/地址放到受控配置表。参数化不代表“任意尺寸都支持”：每种配置需要上界断言、仿真向量和综合结果。限制第 1 版只支持本规格的 256×256、1×1/3×3、batch 1 很合理，但要在接口说明中明确。关键文件与工具版本纳入提交，构建脚本记录综合器版本、器件号、时钟约束和生成的 bitstream SHA256。

CDC、异步复位释放、FIFO 空满边界、计数器回绕、未初始化 RAM、X 传播，以及仿真中偶然通过的阻塞/非阻塞写法，都是板端偶发错帧常因。对每项写断言或测试；不得为了消除仿真中的 X 而把未知值强行替换成 0 掩盖设计问题。

> **自检** 仿真 100 MHz 通过、综合有 1 ns 负 WNS，能否宣称可在 100 MHz 上板？不能；先定位关键路径、加流水或调整架构并重新实现，直到所用约束下时序收敛。

第 11 章 PS/PL、AXI 与 DMA：把正确算子变成完整系统

11.1 三类 AXI 不要混用

AXI4 memory-mapped 访问 DDR 等地址空间，适合批量读取/写回张量；AXI4-Lite 通常供 PS 读写少量控制/状态寄存器；AXI4-Stream 是按拍传输、无内存地址的单向数据流。Zynq 的 AXI HP 端口给 PL 主设备提供通往 DDR/OCM 的高带宽路径；这说的是**能力**，实际带宽取决于突发长度、仲裁、时钟和布局。[S2][S5]

用一张 256×256×3 INT8 输入作例子，裸数据是 196,608 字节；若 DMA 数据宽度为 64 位、传输对齐为 8 字节，首地址、每行跨度和缓冲区地址须符合双方冻结的规则。更复杂的 NCHW 多通道、分块 halo，可能让一次连续传输不够，需要逐段描述符或地址生成。DMA 是否启用 scatter-gather、DRE、长度上限、4 KB 边界等，取决于具体 IP 配置；不要从别人的示例直接抄寄存器常量。[S6]

11.2 Stream 中的 valid、ready、last

AXI4-Stream 每拍仅在 TVALID && TREADY 成立时传输。TDATA 为数据；TLAST 在很多 DMA 配置中标记包尾，但“包”究竟是一整帧、一行或一 tile，应由 B/C 文档明确。若宽度 64 位且每拍放 8 个 INT8，最后不足 8 个字节时用 TKEEP 描述有效字节；接收端需按包协议处理。不要凭 TLAST 自动推断 256 行的位置，更不要让被背压暂停的最后一拍提前改变 TLAST。[S5][S6]

建议为每帧定义状态机：IDLE→CONFIGURED→RUNNING→DONE/ERROR。可定义示意寄存器 version/frame_id/input_addr/output_addr/byte_count/start/status/error/irq_ack，但这些名字和地址**不是本项目现存协议**，必须与 C 组共同审定后冻结。start 时锁存所有参数；运行中写入不改变本帧。超时或复位的行为要写清：哪些 DMA 停止、哪些缓冲可重用、下一帧如何恢复。

11.3 缓存一致性和缓冲区所有权

PS CPU 可能在缓存里保存了新输入，而 DDR 仍是旧内容；把地址交给 DMA 前需要适当 flush。PL 写完输出后，CPU 读前需适当 invalidate，或使用所选运行环境支持的一致性缓冲策略。这里的具体 API 属于 C 组/操作系统环境，但 B 必须与 C 约定内存一致性责任。若读到上一帧，先核对帧号/输入哈希和缓存同步，不应立即怀疑卷积。[S2][S6]

缓冲区分配要防重叠：输入、输出、skip、权重、描述符分别标地址/长度/对齐/所有者/何时释放。双缓冲时，PS 只能写空闲输入，PL 只能读已提交输入；输出在 PL 标记完成前 C 不得读取。同步可用中断或轮询，但不能只靠固定 sleep 毫秒。

11.4 一个 B/C 接口冻结表

字段	建议记录	验证手段
模型包	spec/权重/偏置 SHA256、输出尺度	启动拒绝错版本
输入	RGB→INT8 方式、NCHW/搬运顺序、对齐	固定帧输入 hash
输出	logit、shape、尺度、有效区域	与 INT8 软件逐元素对照
控制	寄存器地址/位含义、启动/完成/错误	读写与复位测试
DMA	端口、宽度、包边界、超时/缓存策略	随机长度/背压测试
帧	frame_id 生成者、回显、错误时处理	连续 100 帧错帧检查

> **自检** 若第 1 帧正确、第 2 帧显示第 1 帧掩膜，优先查缓冲区所有权、DMA 完成等待、CPU 缓存同步和帧号，而非修改权重。

第 12 章 验证：从一个乘加到整帧的证据链

12.1 参考模型、被测设计、比较器

验证环境应包含确定性的输入生成/加载、独立的软件参考、DUT（被测 RTL）、scoreboard 比较器和失败日志。黄金向量是可信输入与预期输出的文件，不等于“RTL 已正确”。给每个比较加 (层,通道,y,x,帧号,期望值,实际值,输入哈希,参数哈希)，遇到首个失配立即保存小范围波形；只打印“测试失败”不够定位。若用 Python 生成期待值，应直接调用经过复核的 quant_math.py，避免把 RTL 中同一个错误复制到测试器。[P1]

第一关只做首层 8×8：逐个输出比较 INT32 acc_golden，**每个元素全等**后再比较 INT8 output_golden。若 acc 不一致，查输入/权重布局、边界填零、偏置小端、有符号扩展、卷积次序；acc 一致而输出不一致，查 Q24 乘法宽度、负半格、舍入、饱和。然后在完整 256×256 单层比 input_full 与 output_full_golden。绝对不能把小块边界误当完整图边界。[P1]

12.2 必备的定向测试集

类别	具体输入	要抓的错误
数值	0、±1、±127、偏置、负半格/正半格、饱和	有符号/位宽/舍入
边界	1×1、左上/右下、核越界、tile 交界	padding/halo/地址
流控	随机 ready=0 连续几十拍、FIFO 满/空	丢拍/重复/标签错位
时序	reset 在中途、启动重复、超时恢复	状态机僵死/脏帧
数据	错版本/错哈希、长度不符、-128	错包未拦截
网络	Pool 同值、Concat 两路尺度异、skip 覆盖	算子顺序/生命周期

随机测试应记录种子；先固定种子复现缺陷，再把修复后样本加入回归。覆盖率不是“随机跑了一万拍”：要确认每种握手/边界/饱和/错误分支至少触发一次。可写 SystemVerilog assertion：当 valid && !ready，下一拍 data/last/frame_id 稳定；每帧接受输入数/产出数符合声明；错误状态最终可被主机读取。

12.3 整网、精度和板端四层验收

第一层是**算术**：各层 acc/INT8 输出与 A 的黄金向量逐位相等。第二层是**网络**：包括 ReLU、Pool、Upsample、Concat 和层间尺度转换的完整单张图在每层检查点一致。第三层是**硬件**：综合、布局布线、时序和资源过关，bitstream 能加载。第四层是**应用**：固定输入帧从 C 端进入、B 加速、C 展示，板端输出与相同模型 INT8 软件参考逐像素比较，再在有真值的合法数据上按相同评估口径算 Dice/IoU。任何一层失败都不能用更高层的漂亮截图遮盖。

软件 INT8 与 FP32 本来有量化差异；板端 INT8 的直接数值参考是同版本软件 INT8，而非 FP32。若板端逐位完全一致，精度应继承同一预处理和后处理口径的 INT8 软件结果；若不一致，要先定位字节级差异，不能直接在测试集“调阈值弥补”硬件 bug。[P1]

> **自检** 小块 acc 全过、完整层输出在右边缘首次错。优先验证小块与全图的 padding 区别、行跨度、DMA 最后一个 burst、x 计数和 TLAST；不要直接更改 q24 舍入逻辑。

第 13 章 综合、实现与真实时序

13.1 四张报告代表四个不同阶段

RTL 仿真检查功能，但不回答能否在特定器件时钟下运行。综合把 RTL 变成器件资源网络，给出初步 LUT/FF/BRAM/DSP 利用量；布局布线后时序报告才更接近实际路径。bitstream 是配置 PL 的产物；板端加载成功后，还需真实数据、长期运行和功耗/延迟记录。每项报告记工具版本、目标 XC7Z020 器件/封装/速度级、顶层、XDC、约束频率、代码 SHA256。UG901 描述综合流程，UG906 描述时序分析；要用适配 Zynq-7000/7 系列的版本和方法。[S3][S4]

13.2 资源率不是唯一 KPI

资源表至少列 LUT、FF、BRAM 18/36Kb 等效块、DSP、IO/BUFG（实际约束相关）和关键 IP 占用。若 BRAM 85% 看似“未满”，仍可能因端口组织、布局拥塞和未来 C 组组件无法实现；要留明确余量。推断 LUT RAM 与 BRAM 的差别、DSP 推断失败、寄存器复制，都要查看综合层级报表，而不是只看总数。

时序设目标后看 WNS/TNS、最差路径的起点/终点、组合逻辑和路由延迟。乘加树长可做流水；高扇出控制信号可做局部寄存/重构；BRAM 到 DSP 路径可能需调整布局或增加寄存。不要只通过调低时钟掩盖原定帧率没达到。若跨时钟，先检查 CDC 是否正确，不能用 false_path 把真实同步问题藏起来。[S4]

13.3 性能分解与 Amdahl 直觉

记录 T_total = T_preprocess + T_H2D + Σ T_layer + T_D2H + T_postprocess + T_display。B 能优化的是其中 PL 和数据搬运的相关部分；若 C 的图像解码/显示占主导，单独把 MAC 加速十倍也无法让总体十倍。相反，若 DMA 占 80%，提高 PE 数没有意义。读写字节、吞吐、busy/idle/blocked 周期计数器可以帮助分解。板端测试重复多帧，报告中位数、分位数、样本量和测量方式，而非挑最快一帧。

功耗可测板级电流/电压及不同状态，但需注明供电方式、仪表误差和是否包含 PS/HDMI 等。Vivado 估计功耗只是在输入活动率等假设下的工具结果，不能标成板端测量。没有实验条件时如实写“未测”。

> **自检** 综合资源过关、WNS 为负，结论是“还没有满足目标时钟”；WNS 非负而实板随机错帧，重点查功能、CDC、DMA 和复位，而不是重做资源表。

第 14 章 上板调试和故障定位

14.1 从最小闭环开始

先确认板卡能加载一个已知 bitstream、PS 能访问控制寄存器、DMA 能搬运不经过卷积的固定字节序列；再接首层小块、完整层、整网。若直接整网首跑，任何错都可能来自参数、缓存、DMA、RTL 或显示，定位成本极高。每个阶段保留可回退的 bitstream 和对应 hash。

硬件读回路径应支持在至少一个固定层输出保存小段片上/DDR 数据，用于与 golden 比较；高吞吐版本可关闭该功能。ILA 等内部逻辑分析器可看握手和计数，但它也消耗资源并改变布线时序，调试版的报告不能直接代替最终版报告。

14.2 用“首个失配”缩小搜索空间

若 DMA loopback 就错：先查地址/对齐/包尾/缓存。Loopback 对而首层 acc 错：查 NCHW/OIHW、signed、padding、权重偏移。Acc 对而 INT8 错：查 Q24 与半格。所有 Conv 对而全网错：查 ReLU、Pool、Upsample、Concat、skip 和换层尺度。整网字节对而显示不对：查 C 的阈值、valid 区、回映、帧号。只在失配类别对应层级修改代码，避免一次改五处使因果不清。

症状	首先排查	保留的证据
整帧重复上一帧	缓存、DMA 完成等待、双缓冲所有权	帧号/输入哈希/输出哈希
边缘一圈错误	padding、tile halo、行跨度	首错坐标与窗口
每通道成比例偏差	input/output scale、weight channel 次序	层配置和原始 acc
负值才出错	signed 扩展、ties-even、算术移位	负半格定向向量
偶发死锁	valid/ready、TLAST、FIFO 满空、超时	ILA/周期状态
只有换模型后错	版本、偏移、旧 bitstream 或旧参数缓存	四类哈希

14.3 错误处理也是功能

应设计启动时版本不符、地址越界、DMA timeout、非法 shape、重复 start、PL reset 中断和 PS 程序退出的可观察行为。控制寄存器应在错误后保留错误码和故障阶段，直到主机确认清除；软复位应让所有有状态算子回到一致边界，不能只把顶层 busy 清零。若丢了一帧，C 应能按帧号避免展示旧结果。

自检 复位在第 200 行发生，重新启动同一帧后前 200 行是旧数据。优先检查输出缓冲重用/写回有效范围，以及 line buffer、layer controller 和 DMA 是否一起复位。

第 15 章 不依赖本书作者，独立做完 B 组的路线图

15.1 八个关卡与退出条件

关卡	你的动作	保留产物	过关门槛
G0 规格	与 A/C 冻结版本、输入、输出、时钟与帧目标	接口表、四类 hash	无未答的字节歧义
G1 纸算	逐层形状/MAC/字节/存活期/位宽	预算表、内存图	与 JSON/脚本一致
G2 数值核	RTL 小块 acc、Q24	仿真日志/波形	黄金向量逐位全等
G3 单层	首层完整 256 图，随机背压	全层首错报告	全部元素一致
G4 整网	非卷积/skip/换层尺度接入	逐层 diff	每层与软件一致
G5 实现	综合、布局、时序收敛	资源/时序/bitstream hash	目标器件及约束达标
G6 上板	loopback→单层→整网	帧日志/逐位对照	固定帧与软件一致
G7 联调	C 端预处理与展示、压力与恢复	协议、时延/稳定性/精度	约定条件全部通过

每关失败就留在此关，保存可重现的输入、代码、配置和首错点。G0 与 G1 可并行开展；G2 必须先于 G4；G5 的性能优化可能使 G2-G4 的回归重做；G7 要求 A/C/B 三方同一模型版本。不要以“B 人少”为由跳过版本和数值门槛。以上是**行动指南**，不表示项目已经走完这八关。

15.2 每周与同学沟通什么

给 A 提问：当前 B 可用的冻结 spec/hash 是什么？层间尺度和 golden 的语义有无变化？若换模型，15 层结构/算子是否仍兼容？给 C 提问：DDR 张量地址由谁分配？帧号/版本怎样传？DMA 包边界、缓存同步、超时由谁负责？显示前阈值/valid/回映在哪执行？三方都应维护变更记录：旧接口、改动原因、新 hash、需要重跑的回归和批准人。

在 B 同学已做部分工作时，你的协助顺序是：先读他已有接口和日志，再跑现有小块 golden；帮忙定位第一个客观失配、补齐资源/时序/帧证据；最后才建议架构变更。不要仅因教材举了某种 PE 排列，就要求推倒已验证的现有方案。

15.3 独立完成能力的最终自测

给你一个更新版 model_spec.json，不依赖作者帮助，应能做到：解析每层 shape/offset/scale；生成逐层预算；证明加法与重定点位宽；画 DDR/BRAM/tile 数据流；为每个 RTL 模块写接口和断言；从小块 acc 到全网逐层通过；建立 AXI/DMA 与 C 的控制协议；让实现时序收敛并按目标测板端延迟；证明板端 INT8 与相同模型软件 INT8 一致；在模型升级时自动触发完整回归。任何一步没有证据，就如实标为未完成。这比背诵某一份 RTL 源码更接近真正独立闭环。

> **过关标准** 你能在不看答案的情况下说出“首层某像素输出错 1”至少五种可能原因，并写出用哪一种黄金文件或波形区分它们。

第 16 章 从空目录动手：工具流程与实验记录

16.1 第一次打开 Vivado 前

确认 PYNQ-Z2 的器件是 xc7z020clg400-1 对应的 Zynq-7000 目标，记下板卡版本、供电、SD 系统和所装 Vivado 版本。项目目录至少分 rtl/、tb/、scripts/、constraints/、reports/、build/、vectors/；版本控制源码、测试、约束和构建脚本，**不要**把庞大临时实现数据库当成唯一交付。存入 spec/参数的哈希和黄金向量来源，避免未来拿错模型。PYNQ 官方站点有板卡系统镜像入口，但下载镜像只用于真正上板；本教材不要求现在下载。[S1][S7]

16.2 先跑一个测试平台

把 input.bin、weight.bin、bias.bin、acc_golden.bin、output_golden.bin 放在可访问路径，写 testbench 按配置读入，驱动一个 3×3 小核模块。先确认仿真器能表示 signed INT8 与 INT32；再加随机背压和 reset。以下是**命令形态示例**，文件列表、顶层名和 Vivado 版本须按自己的项目改，不是本项目已有的可直接运行脚本：

```text
xvlog -sv rtl/mac_lane.sv tb/tb_mac_lane.sv
xelab tb_mac_lane -s mac_sim
xsim mac_sim -R
```

不要只看仿真退出码；在 testbench 中明确 $fatal 首个失配，最后打印比较元素总数。保持至少一份“错误注入”测试：有意颠倒两个权重通道，确保测试器真的会失败。若失败用命令能在同样版本复现，再着手修 RTL。

16.3 一个独立 PL 顶层的批量实现模板

复杂 PS/AXI 系统可用 Vivado Block Design、生成 wrapper 和官方 IP；下面 Tcl 仅示意**独立 PL 顶层**的实现与报告顺序，并非一键构建本项目整板设计。真实脚本要显式纳入所有 RTL/IP、所选器件、时钟、输入/输出约束、许可与版本。[S3][S4]

```tcl
read_verilog -sv rtl/my_top.sv
read_xdc constraints/my_clock.xdc
synth_design -top my_top -part xc7z020clg400-1
report_utilization -file reports/post_synth_util.rpt
opt_design
place_design
route_design
report_utilization -file reports/post_route_util.rpt
report_timing_summary -file reports/post_route_timing.rpt
write_checkpoint -force build/routed.dcp
```

例如 create_clock -period 10.000 [get_ports clk] 只在顶层 clk 确为 100 MHz 的真实输入时适用；Zynq PS 提供给 PL 的 FCLK、Block Design 输出与时钟命名可能不同，务必查看约束是否覆盖了实际时钟路径。约束没覆盖到的路径“没有违例”没有可信度。实现报告中看到负 WNS，就回到最差路径分析；即使时序通过，仍需把 bitstream 与板级测试绑定。[S4]

16.4 实验记录表要能让别人复现

每次提交记录日期、目标/器件、代码 commit 或目录哈希、模型 spec/权重哈希、RTL 参数、Vivado 版本、时钟/XDC、随机测试种子、通过/失败的 golden 元素数、综合/实现资源、WNS/TNS、板端输入输出哈希、耗时分布。更换 P 或 tile 时新建记录行；不要覆盖旧结果。任何指标旁标“估计/仿真/实现后/板端”。这样既帮助自己判断优化，也方便 B 同学与你分工时互相信任。

16.5 第一次建立 Zynq Block Design 的实际顺序

先在 Vivado 建 RTL 项目并选择与板卡相符的 Zynq-7000 器件/board file；进入 IP Integrator 建 Block Design，加入 ZYNQ7 Processing System，按板卡配置 DDR/FIXED_IO、FCLK、复位和 GP/HP AXI 端口。随后加入 AXI DMA 与自定义 RTL/IP：PS 的 M_AXI_GP0 通过互连访问 DMA 和加速器的 AXI-Lite 控制口；DMA 的 memory-mapped 主口经 PS 的 S_AXI_HP0 等可用高性能口访问 DDR；DMA MM2S 流接加速器输入，S2MM 流接输出。再连接同一时钟/复位域或设计明确的 CDC，检查中断连接与地址空间。使用 Validate Design，生成 HDL wrapper，设置真实 XDC，综合、实现并生成 bitstream。IP 名称、自动化选项和连线方向应以所装版本与 UG994/UG585 为准，不能照此段当自动生成的图已经验证。[S2][S8]

Block Design 连接	目的	自检
PS M_AXI_GP0 → 控制口	PS 写启动/读状态	地址映射可读回

Block Design 连接	目的	自检
DMA M_AXI_MM2S/S2MM → PS S_AXI_HP*	从/向 DDR 搬帧	loopback 字节相等
DMA M_AXIS_MM2S → B 输入	流入 INT8	TKEEP/TLAST 合同
B 输出 → DMA S_AXIS_S2MM	流出 logit	输出长度/帧号一致
FCLK/reset/irq	同步/中断	复位恢复、无悬空

PYNQ Overlay 通常需要**同名 stem 的 `.bit` 和 `.hwh`**；HWH 是 Vivado Block Design 生成的硬件交接信息，PYNQ 用它识别 IP、地址、时钟等，不是模型权重。C 可用 Overlay('b_accel.bit') 加载，查看 ip_dict 与实际 IP 名称；B 应提供 bitstream/HWH 成对哈希、寄存器表和版本，而非只给 .bit。[S9]

PYNQ 的 allocate 提供适合 DMA 的连续缓冲，PynqBuffer 公开物理地址及必要的 flush()/invalidate()；C 在非一致缓存配置下提交输入前、读取输出前按文档同步。DMA 常见示例用 sendchannel.transfer、recvchannel.transfer 与 wait()，但 B/C 必须核对实际连接、包尾与 IP 配置。PYNQ DMA 文档示例指出默认最大传输长度可能是 14 位（约 16 KiB）；本项目单张原始 INT8 RGB 输入已有 **196,608 字节**，因此需在 Vivado 配置足够的传输长度或按双方冻结的协议分段。若直接复制五个 uint32 的入门例子搬全图，会在长度边界失败。[S10][S11]

16.6 如何从初学者教程升级到本项目

教程通常只展示一个 5 元素 loopback 或一个矩阵乘法核。迁移到本项目时，逐项扩展：数据从 uint32 改为有符号 INT8 并明确每拍打包；单一缓冲变成输入/输出/权重/skip 多缓冲；单个函数调用变成 15 层与非卷积算子的时序；一次成功改为错帧、超时、复位和连续压力测试；最终用 A 的黄金向量验算，而不是用教程自己的常量。保留最小 loopback 作为每次改 Block Design 后的回归，这能把接口问题与网络算术问题分开。

> **自检** 如果只有一个截图显示“Implementation Complete”，但拿不出 XDC、WNS、模型包 hash 和逐位输出记录，仍不能宣称 B 任务完成。

第 17 章 八个实训：局部可复现，不偷跳整网验收

本章把前面知识拆成八个小实验。它们足以训练你独立搭建完整 B 组工程，却**不是**一份已有的整网 RTL 交付。每个实验都写“准备、动作、过关、失败时先查什么”；请自行给文件命名并记录版本。若 B 同学已有等价模块，优先用这些练习审查与补强，而非另起一套平行代码。

17.1 实训一：从二进制文件读出正确的数

准备：冻结基线 model_spec.json、首层 config.json、首层 input.bin/weight.bin/bias.bin。动作：编一个小型读取器，逐个断言文件 SHA256、长度和偏移；输出 input[0,0,0,0]、weight[0,0,1,1]、bias[0]，再将 NCHW/OIHW 的地址公式写在纸上。首层 weight[0,0,1,1] 由真实包读到 -102，偏置 bias[0]=-18585；若读成 154 或怪异大数，很可能是 signed 或小端错误。首层权重文件有 216 字节、偏置文件 32 字节，整包偏移以 JSON 为准。[P1]

过关：文件校验全部通过；对任意三个随机索引，用“手算偏移→读单字节”和数组索引两种方法得同一数。故障定位：哈希不同先停，不继续比较数值；哈希相同而数值不同，查 dtype、reshape 次序、小端与有符号。延伸：读取第二层时偏置起始偏移是 32 字节，验证你没有把它误当第 32 个 INT32 元素。

17.2 实训二：一条 3×3 流如何生成窗口

准备：先用 5×5 单通道人工图，像素值 1…25，padding=1，stride=1；再用首层 8×8 输入。动作：每拍喂一个像素，维护行/列计数和两行历史；输出窗口及 (y,x) 标签。手算左上、上边中央、中心、右下四个窗口。左上角的顶/左元素必须 0；x=0 时上一行末尾永远不充当左邻居。暂停输入 1、2、17 拍测试窗口保持与计数器不前进。

过关：25 个输出坐标各出现一次；窗口 9 个值与软件 pad+slice 全等；施加背压后结果序列不变。失败时看：行尾第一个错通常是行计数/换行；所有边缘错而中心对通常是 padding；停顿后错通常是 valid/ready 处理。扩展到 Cin=3 时要明示输入是 NCHW 整通道连续，或在入口重排为像素交织流，并证明重排后的窗口仍对应同一个 (c,y,x)。[P1]

17.3 实训三：一个 MAC lane 到两个 PE

准备：实训二窗口、首层权重/偏置、acc_golden.bin。动作：先写单 lane 按 27 项顺序串行点积，start_dot 装偏置，之后每接受一对 x/w 做一次乘加；第 27 次完成后输出 INT32。再复制为两个 lane 同时计算两个输出通道，保证同一输入窗口广播、不同权重通道不串线。将耗时分成“27 拍理论计算、窗口形成、权重读取、输出堵塞”四项；记录每一项实际拍数。

过关：独立 8×8 小块所有首层输出通道的 acc 与 acc_golden.bin 全等；随机暂停时不丢数据。失败时：只第 0 通道对而第 1 通道错，查权重通道 offset 和各 PE 偏置；第 26/27 项边界错，查

start_dot 优先级与最后一乘积入账。再把 lane 改为同拍 3 或 9 乘法，验证吞吐增益与资源报告是否相符，切勿只靠代码里 for 的次数声称并行。

17.4 实训四：把 Q24 舍入写成可证明的整数电路

准备：固定 shift=24，手工生成 p=k*2^24 ± 2^23 的正负边界向量，外加饱和到 ±127 的例子。动作：先用 Python 整数商/余数写独立参考；再在 RTL 中做 acc × m 的足宽有符号积、绝对值/余数/半格判断/还原符号/饱和。对 p=2.5*2^24、3.5*2^24、-2.5*2^24、-3.5*2^24 强制比较。对 INT64_MIN 等可能的取绝对值例外明确不可达证明或特殊处理，不允许无定义溢出。

过关：定向边界全对，再用随机积至少数千组与项目 round_divide_even 比较；最后通过全部层小块 output_golden.bin。失败时：acc 正确而输出固定偏 1，优先查半格/截断；负数错而正数对，优先查有符号位移和绝对值；大值错，查乘法中间宽度与饱和。对本包计算的累加绝对值保守上界最大为 13,936,910，Q24 最大乘数 177,073，**不同层两个最大值相乘是保守总上界** 2,467,850,464,430，已远超有符号 32 位；选 64 位中间积便于合同清晰。换包应重算，而非复制上界。[P1]

17.5 实训五：非卷积与 skip 的合成陷阱

准备：为 ReLU、2×2 pool、nearest upsample、Concat 准备 2×2 或 4×4 手工整数阵列，给两条 Concat 输入各一个不同尺度。动作：逐项写出空间索引变化；Concat 的前半通道来自编码器，后半来自上采样；把两条路按各自尺度转换到下层入口尺度。先用软件手算一个像素的两个通道，再跑 RTL。切换一个错误版本（把 Concat 次序颠倒），确认单元测试能识别。

过关：人工阵列和真实完整层 golden 之间可逐层追踪；在 output_full_golden 匹配后，下层 input_full 也匹配。失败时：池化只在奇数列错，查 2×2 分组；Concat 后所有输出通道错，查拼接次序与尺度；第一个 skip 正确、后两个错误，查 DDR 存活期和覆盖地址。若为速度把非卷积算子融入卷积写回，也要保留可打开的中间比较点。[P1]

17.6 实训六：ready/valid 和 AXI-Stream 背压

准备：写一个源、一个有 1-2 级 FIFO 的 DUT、一个随机 ready 的接收器。动作：源送 37 个标号递增字节，每 8 字节打包，最后一拍用 TKEEP 标有效字节并在约定的包尾置 TLAST；接收器以固定种子随机停顿。每次 TVALID && TREADY 才更新“已传输字节数”。插入 reset、重复 start、末拍卡住、连续 20 拍背压四种情况。

过关：传出的字节序列、TKEEP、TLAST、帧号与期望完全一致，无缺号/重复号；断言 valid && !ready 时负载稳定；reset/错误后能按协议重新开始。失败时：停顿后序列缺字节，查上游是否错误递增；末拍提前结束，查 TLAST 是否跟随数据经过 FIFO。这个实训先不接 DMA，以免协议错误与 DMA 配置错误混在一起。[S5][S6]

17.7 实训七：DDR/DMA loopback 后再上单层

准备：与 C 冻结一帧输入/输出缓冲、对齐、长度、帧号、cache flush/invalidate 责任；先使用 PL 原样回传，不做卷积。动作：PS 生成伪随机字节并记录 SHA256，提交给 DMA/PL，等待完成或超时，读回并比较；改变长度、对齐与是否跨 4 KB 边界，在**当前 DMA IP 配置允许范围内**测试。再把 loopback 变为首层完整张量，比较 output_full_golden。[S6]

过关：规定长度范围内 loopback 逐字节全对，连续至少 100 帧帧号一致；超时有错误码且可恢复；首层输出逐位全等。失败时：第一帧错查参数和 cache，第二帧错查 buffer 复用，靠近 4 KB 位置错查突发边界/描述符。不要通过让 CPU 直接 memcpy 输出伪装成 PL 完成；记录实际 DMA/PL 状态位与耗时。

17.8 实训八：用一次受控优化完成工程回归

准备：保存一版已通过 G2-G6 的可重现版本。动作：只修改 P_out、tile 宽度或一处流水级中的一项；先重跑数值/随机背压，再综合/实现并记录资源、WNS、时钟，再上板测同一批固定帧和总时延。把“优化前/后”放同一表，逐项注明有没有提高、有没有牺牲资源、是否改变 DDR 字节数、有没有不良尾部延迟。若 WNS 从正变负，哪怕仿真更快也不算完成优化。

过关：功能无回归，约束全通过，板端目标指标有统计学上可见的改进，且留有退回旧版的方法。失败时先回退单一改动，比较首个失配或最差路径，不要把多个结构变化混在一次实验里。这个习惯能把“凭感觉调架构”变成可检验的工程论证。

第 18 章 三张计算纸：地址、位宽、吞吐

18.1 地址纸：让每个字节都有名字

连续 NCHW INT8 的一维偏移为 ((n*C+c)*H+y)*W+x。首层输入 [1,3,256,256] 中，input[0,1,0,0] 的偏移是 65,536 字节；input[0,2,128,128] 是 2*65,536+128*256+128=163,968 字节。首层权重 OIHW [8,3,3,3] 中，weight[1,2,1,2] 的偏移为 (((1*3+2)*3+1)*3+2)=50 字节。把“下标→偏移→二进制字节”手算正确后再优化 DDR 交织。若内部格式改成 NHWC 或打包 64 位，必须写转换映射和逆映射测试，不得再把原始 golden 字节直接按新顺序读取。[P1]

偏置包采用小端 INT32：第 0 层 8 个偏置占 32 字节；第二层从偏移 32 字节开始，而不是“索引 32”。每层 weight_offset_bytes/weight_nbytes/bias_offset_bytes/bias_nbytes 都由 spec 给出。对偏移、长度做 0≤offset、offset+length≤文件长度、相邻区间不意外重叠的验证。若按 DMA 宽度填充权重，需要新内存布局版本，仍保留对原包的映射与哈希。

18.2 位宽纸：为什么“它是 INT8 网络”仍可能需要 64 位

输入和权重各在 [-127,127]，单次乘积绝对值最多 16,129；Cin*K*K 项相加和 bias 需要足够累加位。对冻结包逐层计算保守上界，decoder_blocks.0.0 最大约 13,936,910，远小于 INT32 上限 2,147,483,647；但你仍应按 INT32 合同输出，避免未来换模型时隐式变窄。Q24 乘数乘上累加值的乘积可能达到万亿量级，故应先用 64 位或经证明足够的位宽，再做商/余数和饱和。[P1]

位宽证明要同时涵盖“静态算术最大值”和“RTL 表达式的求值宽度”。例如把两个 8 位 signed 直接相乘后赋给 32 位目标，在不同语言规则/表达式环境下可能先发生窄位求值；明确把操作数扩展并检查合成网表。一个断言可在仿真中检查 accumulator 是否超过协议范围，但断言不能代替数学证明。选择饱和还是回绕必须由 A 的数值合同决定，不能因硬件省资源自行改变。

18.3 吞吐纸：算得快与送得上是两回事

方案甲假设 16 MAC/拍、100 MHz、利用率 0.4，计算时间约 675,282,944/(16*100e6*0.4)=1.055 秒；方案乙 32 MAC/拍、100 MHz、利用率 0.5，约 0.422 秒。这两行只是**假设**。若方案乙需要更多并行权重端口，使 DDR 搬运从 0.2 秒变到 0.8 秒或时序只到 60 MHz，它可能总体更慢。给每层记录 T_compute,T_read,T_write,T_control，对于可重叠阶段近似取最大值、串行阶段求和，并用板端计数器校正模型。

Roofline 思想可以简化为：持续吞吐同时受“乘加能力”和“外存字节能力”限制。提高算力前先算 MAC/DDR_byte；它越低，越容易被带宽卡住。对于重复访问窗口/权重的朴素方案与 line-buffer 方案，各写一遍 DDR 字节预算。即使理论带宽够，短 burst、随机访问、PS 竞争和 DMA 启动成本仍会降低有效带宽，因此实际测量不可省。[S2][S6]

> **总复习** 一套可靠的 B 设计至少同时拿出三张纸——字节地址图、位宽证明、计算/访存时间模型。任何一张缺失，后续性能数字都可能是建立在错误数据上的幻觉。

附录 A 算术、预算与数据读取练习

A.1 纸笔题与答案

题 1：4 位补码 1101 是多少？答：最高位为负，1101 取反加一得 0011，因此为 -3；若被错当无符号数则为 13。题 2：Cin=8,Cout=16,K=3,H=W=128 的卷积有多少 MAC？答：128*128*16*8*9=18,874,368 MAC。权重 16*8*9=1152 字节，偏置 16*4=64 字节（INT8/INT32）。

题 3：acc=-5,m_q24=2^23，输出应为多少？答：-2.5 恰好半格，两个候选 -2 与 -3 中偶数是 -2；若直接算术右移可能得 -3，故不等价。题 4：3×3 卷积输出 64×64 tile，Cin=16，完整 halo 输入最少多少 INT8 字节？答：(64+2)*(64+2)*16=69,696 字节，未计通道并行权重/输出/FIFO。题 5：若 32 MAC/拍、100 MHz、理想利用率 1，下界为 675,282,944/(3.2e9)≈0.211 秒；利用率 0.5 时约 0.422 秒，尚未计搬运。[P1]

题 6：首层 [1,3,256,256] INT8 输入多少字节？答：196,608。首层输出 [1,8,256,256] 多少？答：524,288。问：两者加起来是否就等于网络最大工作内存？答：不是，skip、双缓冲、权重和后续层也有存活期。题 7：模型输出字节 0，在旧基线阈值 0.5 下是前景还是背景？答：logit=0，sigmoid=0.5，条件 >=0.5，因此是前景；非补边区域以外仍强制背景。[P1]

A.2 真实首层向量的可复算路径

本项目在 docs/b_textbook/verify_examples.py 提供只读核查脚本；使用项目 Python 环境运行，输出应包含 layers=15、macs=675282944、first_layer_channel_sums=[-4261,21622,-2698]、acc=-3922、multiplier_q24=13370、requant=-3、golden=-3。脚本读取冻结交付包，而非训练数据或未来候选；读取失败时先检查项目相对路径与交付包版本，不要改黄金向量使结果“通过”。

```powershell
& .\.venv\Scripts\python.exe docs/b_textbook/verify_examples.py
& .\.venv\Scripts\python.exe software/role_a/verify_export.py `
  --spec deliverables/role_a/int8/model_spec.json
```

第一个命令核对教材算例和保守位宽上界；第二个命令核对交付包的 15 层哈希与黄金向量。教材复算得到所有层的保守最大累加绝对值上界为 **13,936,910**，发生在 decoder_blocks.0.0；它小于有符号 INT32 上界，但此推导依赖当前包的实际 bias 和 [-127,127] 输入/权重，换包必须重算。中间 acc*m_q24 仍需独立证明可放入所选宽度。[P1]

A.3 你自己写一个最小比较器

先读 JSON 的 input_shape/weight_shape/output_shape 和 files_sha256；逐文件检查长度、哈希，按小端读取 bias 与 accumulator，按 NCHW/OIHW reshape。用四重循环或可信的 NumPy 参考算 acc，比较第一个不一致的 (o,y,x)；再用整数商/余数实现 ties-even，不要先浮点除法后转回整数。输出一份只含版本、元素数、首错点的报告。拿首层小块过关后，换另一层 1×1、不同通道数，验证比较器不只“适合首层”。

若自编比较器与项目 verify_export.py 冲突，先比输入文件哈希和边界语义，再检查你的实现；不要直接把“多数票”当正确。A 组参考代码和交付包 hash 是当前合同，任何真实歧义须回到 A 组澄清。

附录 B 接口冻结、验证与交付模板

B.1 接口冻结记录（复制后填写）

项	内容（示例位留空）
冻结编号/日期/负责人	_____
模型 checkpoint/spec/weights/bias 哈希	_____
输入：shape、dtype、有效值、布局、尺度	_____
DDR：地址对齐、行跨度、张量顺序、buffer 所有者	_____
DMA：MM2S/S2MM、总字节、TKEEP/TLAST 语义	_____
控制：寄存器地址、start/done/error、复位/超时	_____
输出：shape、尺度、阈值实施方、valid 区	_____
时钟/目标延迟/资源余量/测量方式	_____
A/B/C 三方确认与变更记录	_____

填表前不要把空白地址当成已经存在的寄存器；填后把表与代码、文档的 hash 同时冻结。即使同一个寄存器名保留，位含义变了也要更新接口版本。

B.2 数值和硬件报告模板

验收项	证据路径/数值	状态
文件哈希和 offset 全通过	_____	未测/通过/失败
15 层小块 INT32 acc 逐位一致	_____	未测/通过/失败
15 层小块 INT8 逐位一致	_____	未测/通过/失败
完整单层及整网每层一致	_____	未测/通过/失败
随机背压/复位/超时/错版本	_____	未测/通过/失败
综合与实现资源（含 IP）	_____	未测/通过/失败
目标时钟 WNS/TNS 与约束覆盖	_____	未测/通过/失败
板端 1/100/N 帧的输入输出 hash	_____	未测/通过/失败
同版本软件 INT8 对照与 Dice/IoU	_____	未测/通过/失败
各阶段延迟分布和功耗条件	_____	未测/通过/失败

“未测”是合法状态，“没有证据却填通过”不是。未来模型换版时，复制一份新表，从文件哈希开始重跑，不改旧表。

附录 C 术语、来源与阅读索引

C.1 速查术语

RTL：寄存器传输级描述，说明每个时钟拍数据如何移动/计算。MAC：一次乘法和加法的运算计数。PE：承担若干 MAC 与局部控制的处理单元。BRAM：FPGA 片上块 RAM。DSP：专用算术资源。PS/PL：Zynq 的处理系统/可编程逻辑。DDR：板上外部动态存储。AXI4-Lite：控制寄存器常用总线。AXI4-Stream：ready/valid 流。DMA：不经 CPU 逐字节搬运的传输引擎。Q24：使用 24 位分母的整数比例表示。golden vector：已定义输入下的预期逐位输出。WNS：最差时序裕量；负值是时序未收敛。bitstream：用于配置 PL 的文件。

C.2 项目内一手材料

[P1] deliverables/role_a/README.md、B_C_并行开发交接说明.md、int8/model_spec.json 及其各层 config.json/二进制 golden，另见 software/role_a/model.py、quant_math.py、verify_export.py。这些文件固定了本文的 2016 基线数值合同；教材实算脚本为 docs/b_textbook/verify_examples.py。项目路径以 E:\嵌赛\pld-medical 为根。

[P2] AGENTS.md 与 docs/A02_当前断点.md，说明当前 A 组候选、后处理仍待选择，**不是** B 组已发布硬件版本。未来状态变化时先读新版本，不把本教材日期快照当永久事实。

C.3 官方外部资料

[S1] PYNQ-Z2 User Manual v1.0，板卡器件、BRAM/DSP、DDR 与板上接口：https://pynq.tue.nl/doc/pynqz2_user_manual_v1_0.pdf。

[S2] AMD, Zynq-7000 SoC Technical Reference Manual, UG585，PS/PL、AXI HP、DDR 与时钟/复位：https://docs.amd.com/r/en-US/ug585-zynq-7000-SoC-TRM。

[S3] AMD, Vivado Design Suite User Guide: Synthesis, UG901（所引 2021.1 版），可综合 RTL、综合流程及推断：https://docs.amd.com/v/u/2021.1-English/ug901-vivado-synthesis。

[S4] AMD, Vivado Design Suite User Guide: Design Analysis and Closure Techniques, UG906，时序报告与实现后分析：https://docs.amd.com/r/en-US/ug906-vivado-design-analysis。查与你安装的 Vivado 相同或适配版本，不把新器件专用的优化流程套用到 Zynq-7000。

[S5] AMD, AXI4 Interface Protocol 产品页，AXI4、AXI4-Lite、AXI4-Stream 用途：https://www.amd.com/en/products/adaptive-socs-and-fpgas/intellectual-property/axi.html。

[S6] AMD, AXI DMA LogiCORE IP Product Guide, PG021，模式、描述符、状态与配置相关限制：https://docs.amd.com/r/en-US/pg021_axi_dma。

[S7] PYNQ 官方板卡与镜像入口，PYNQ-Z2 平台信息：https://www.pynq.io/boards.html。此页的“当前镜像”会变化，实际部署时重新核对。

[S8] AMD, Vivado Design Suite User Guide: Designing IP Subsystems Using IP Integrator, UG994，Block Design/IP 连接方法：https://docs.amd.com/r/2025.1-English/ug994-vivado-ip-subsystems。

[S9] PYNQ 3.1 Overlay Design，.bit 与 .hwh 的生成与配对：https://pynq.readthedocs.io/en/v3.1/overlay_design_methodology/overlay_design.html。

[S10] PYNQ 3.1 Buffer API，allocate、物理地址、缓存同步：https://pynq.readthedocs.io/en/v3.1/pynq_package/pynq.buffer.html。

[S11] PYNQ 3.1 DMA 教程，传输/等待和最大长度配置提示：https://pynq.readthedocs.io/en/v3.1/pynq_libraries/dma.html。

本书未从外部来源复制长篇正文、完整设计或图示；外部手册用于核验事实与后续查阅。书中未宣称项目已有 B 组 RTL、bitstream、综合报告或板端性能。若读者要沿某条优化路线实施，应先在本地完成对应算术、资源和时序验证。